

Question: Write server and client side program for stream and datagram to send and receive messages and also explain what's going on under the hood.

Objective

To implement client–server communication using both stream (TCP) and datagram (UDP) sockets and understand the underlying communication mechanisms.

Theory

Socket programming is a method of enabling communication between two machines over a network using the transport layer protocols of the TCP/IP model. A socket acts as an endpoint for sending or receiving data.

1. Stream Socket (TCP)

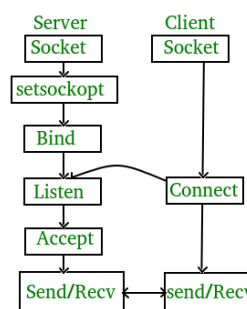
TCP (Transmission Control Protocol) is a connection-oriented protocol that provides reliable communication between two processes. Before data transfer begins, a connection is established using a handshake process. Once connected, data is transmitted as a continuous stream of bytes.

TCP ensures:

- Reliable delivery of data
- Ordered reception of packets
- Error detection and retransmission in case of packet loss

However, TCP does not preserve message boundaries. This means data sent in separate write operations may be received together or split at the receiver side. Because of these features, TCP is commonly used in applications where accuracy is more important than speed, such as web browsing, file transfer, and email services

TCP Server and Client Implementation



The entire process can be broken down into following steps:

TCP Server -

1. using create(), Create TCP socket.
2. using bind(), Bind the socket to server address.
3. using listen(), put the server socket in a passive mode, where it waits for the client to approach the server to make a connection
4. using accept(), At this point, connection is established between client and server, and they are ready to transfer data.
5. Go back to Step 3.

TCP Client -

1. Create TCP socket.
2. connect newly created client socket to server.

2. Datagram Socket (UDP)

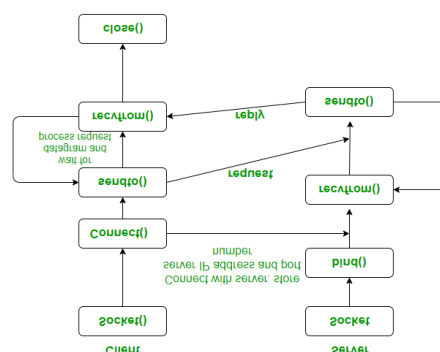
UDP (User Datagram Protocol) is a connectionless protocol that sends data in the form of independent packets called datagrams. No prior connection is established between sender and receiver.

UDP does not guarantee:

- Delivery of packets
- Order of packets
- Duplicate protection

Each datagram is handled independently, which makes UDP faster and more efficient for real-time applications. However, it sacrifices reliability for speed. UDP is commonly used in applications such as video streaming, online gaming, and DNS queries where occasional data loss is acceptable.

UDP Server and Client Implementation



Program:

1. Stream Socket (TCP)

Server

TCP Server Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
```

```
#define PORT 8080
```

```
#define SIZE 1024
```

```
int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    socklen_t addrlen = sizeof(address);
    char buffer[SIZE];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 1);

    printf("Waiting for client...\n");

    new_socket = accept(server_fd, (struct sockaddr *)&address, &addrlen);

    printf("Client connected.\n");

    while (1) {
        memset(buffer, 0, SIZE);

        recv(new_socket, buffer, SIZE, 0);

        if(strcmp(buffer, "exit")==0)
            break;

        printf("\nClient : %s\n", buffer);
```

```

    printf("Server : ");
    fgets(buffer, SIZE, stdin);
    buffer[strcspn(buffer, "\n")] = 0;

    send(new_socket, buffer, strlen(buffer)+1, 0);

    if(strcmp(buffer, "exit")==0)
        break;
}

close(new_socket);
close(server_fd);

return 0;
}

```

Client

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define SIZE 1024

int main() {

    int sock;
    struct sockaddr_in server;
    char buffer[SIZE];

    sock = socket(AF_INET, SOCK_STREAM, 0);

    server.sin_family = AF_INET;
    server.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &server.sin_addr);

    connect(sock, (struct sockaddr *)&server, sizeof(server));

    while(1){

        printf("Client : ");
        fgets(buffer, SIZE, stdin);
    }
}

```

```

    buffer[strcspn(buffer, "\n")] = 0;

    send(sock, buffer, strlen(buffer)+1, 0);

    if(strcmp(buffer, "exit")==0)
        break;

    memset(buffer, 0, SIZE);

    recv(sock, buffer, SIZE, 0);

    if(strcmp(buffer, "exit")==0)
        break;

    printf("Server : %s\n", buffer);
}

close(sock);

return 0;
}

```

Output:

```

subesh@SubeshYadav:/mnt/f/Study/6 th sem/system/lab1$ ./tcp_server
Waiting for client...
Client connected.

Client : Hello!
Server : Hi, How are you?

Client : I'm fine thanks, and what about you?
Server : I'm fine too!, Glad to hear you!!

```

```

subesh@SubeshYadav:/mnt/f/Study/6 th sem/system/lab1$ ./tcp_client
Client : Hello!
Server : Hi, How are you?
Client : I'm fine thanks, and what about you?
Server : I'm fine too!, Glad to hear you!!
Client : exit

```

2. Datagram Socket (UDP)

Server

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define SIZE 1024

int main(){

    int sockfd;
    char buffer[SIZE];

    struct sockaddr_in server, client;
    socklen_t len = sizeof(client);

    sockfd = socket(AF_INET,SOCK_DGRAM,0);

    server.sin_family = AF_INET;
    server.sin_addr.s_addr = INADDR_ANY;
    server.sin_port = htons(PORT);

    bind(sockfd,(struct sockaddr*)&server,sizeof(server));

    printf("UDP Server Started...\n");

    while(1){

        memset(buffer,0,SIZE);

        recvfrom(sockfd,buffer,SIZE,0,(struct sockaddr*)&client,&len);

        if(strcmp(buffer,"exit")==0)
            break;

        printf("\nClient : %s\n",buffer);

        printf("Server : ");
        fgets(buffer,SIZE,stdin);
        buffer[strcspn(buffer,"\n")] = 0;

        sendto(sockfd,buffer,strlen(buffer)+1,0,
            (struct sockaddr*)&client,len);
```

```

        if(strcmp(buffer,"exit")==0)
            break;
    }

    close(sockfd);

    return 0;
}

```

Client

```

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define SIZE 1024

int main(){

    int sockfd;
    char buffer[SIZE];

    struct sockaddr_in server;
    socklen_t len = sizeof(server);

    sockfd = socket(AF_INET,SOCK_DGRAM,0);

    server.sin_family = AF_INET;
    server.sin_port = htons(PORT);
    inet_pton(AF_INET,"127.0.0.1",&server.sin_addr);

    while(1){

        printf("Client : ");
        fgets(buffer,SIZE,stdin);
        buffer[strcspn(buffer,"\n")] = 0;

        sendto(sockfd,buffer,strlen(buffer)+1,0,
            (struct sockaddr*)&server,len);

        if(strcmp(buffer,"exit")==0)
            break;

        memset(buffer,0,SIZE);
    }
}

```

```

        recvfrom(sockfd,buffer,SIZE,0,
                (struct sockaddr*)&server,&len);

        if(strcmp(buffer,"exit")==0)
            break;

        printf("Server : %s\n",buffer);
    }

    close(sockfd);

    return 0;
}

```

Output:

```

subesh@SubeshYadav:/mnt/f/Study/6 th sem/system/lab1$ ./udp_server
UDP Server Started...

```

```

Client : Hello
Server : Hi, How are you?

```

```

Client : I'm fine thanks, and what about you?
Server : I'm fine too!, Glad to hear you!!

```

```

subesh@SubeshYadav:/mnt/f/Study/6 th sem/system/lab1$ ./udp_client

```

```

Client : Hello
Server : Hi, How are you?
Client : I'm fine thanks, and what about you?
Server : I'm fine too!, Glad to hear you!!
Client : exit

```


Under the Hood Explanations

1. TCP – What actually happens inside the OS

A. During connect() and accept()

When connect() is called:

- The OS creates a TCP Transmission Control Block (TCB) entry.
- A SYN packet is constructed in kernel space (not user space).
- Packet is passed to IP layer → NIC → network.
- Socket state becomes: SYN_SENT.

When server calls accept():

- Server socket is in LISTEN state.
- Kernel maintains two queues:
 1. SYN queue (half-open connections)
 2. Accept queue (fully established connections)

Only after handshake completes:

- connection moves from SYN queue → accept queue

accept() simply pops from accept queue

- It does NOT create connection
- It only returns a new file descriptor for an already established kernel connection object

B. During send() in TCP

When you call send():

- Data is copied from user buffer → kernel send buffer
- Kernel does NOT send immediately
- TCP layer performs:
 1. segmentation (based on MSS)
 2. sequence number assignment
 3. buffering in retransmission queue

Then:

- packets are scheduled by TCP congestion control algorithm
- sent via IP layer

C. During recv() in TCP

When you call recv():

- OS reads from TCP receive buffer (kernel space)
- It does NOT correspond to one send()

Kernel behavior:

- packets may already be:
 1. merged (Nagle buffering)
 2. split (MSS segmentation)
- TCP reorders using sequence numbers
- delivers a continuous byte stream

Key internal rule:

TCP reconstructs a byte sequence, not message boundaries

D. Reliability internals

Inside kernel TCP stack:

- every segment has:
 1. sequence number
 2. ACK tracking entry

If ACK not received:

- packet stored in retransmission queue
- retransmitted after timeout (RTO)

Kernel also maintains:

- sliding window
- congestion window (cwnd)
- receive window (rwnd)

2. UDP – What happens inside the OS

A. During sendto()

When sendto() is called:

- OS does NOT create connection state
- It immediately constructs a UDP datagram
 1. adds UDP header
 2. attaches payload

Then:

- packet is passed to IP layer immediately

No buffering logic like TCP stream management

No handshake, no session table update

B. During recvfrom()

- Kernel receives full UDP datagram from network buffer
- Entire datagram is delivered at once

Important:

- If buffer is smaller → datagram is truncated
- If no datagram exists → call blocks

Conclusion

In this lab, TCP and UDP client-server communication was successfully implemented using socket programming in Linux. The TCP program established a reliable connection and allowed continuous two-way message exchange, while the UDP program transmitted messages without creating a connection, demonstrating faster but less reliable communication. The implementation also provided an understanding of the underlying socket operations, including connection establishment, data transmission, and reception performed by the operating system. Overall, the lab helped compare the working principles, advantages, and limitations of both TCP and UDP protocols in network programming.